

Performance Testing Report

Computer Systems and Network Engineering – Final Year
Research Project

Project Title: CargoMate - Smart Robotic Inventory System with Cloud Integration (Robotic Arm Subsystem)

Student Name: B. K. Senara

Student ID: IT22036698

Supervisor: Mr. Udith Dharmakeerthi

Date: 30/04/2026

1. Introduction and Objective

CargoMate is a low-cost robotic inventory handling system designed for small-scale warehouse environments. The subsystem developed in this project focuses on an embedded robotic arm capable of performing automated pick-and-place operations based on item attributes decoded from QR codes. The system integrates servo motor control, a rule-based sorting algorithm, and real-time cloud synchronization using Firebase.

Performance testing was conducted to ensure that the system meets key operational requirements under realistic working conditions. These include placement accuracy, system responsiveness, calibration consistency, and cloud communication efficiency. Since the system is designed for practical deployment in resource-constrained environments, reliability and consistency are critical.

The main objectives of testing were:

- To validate placement accuracy across different object weights
- To measure cycle time and system throughput
- To evaluate servo calibration repeatability
- To assess real-time slot management and cloud latency

The system was expected to achieve at least 90% placement accuracy, maintain a throughput of 5 objects per minute, and ensure responsive cloud updates within acceptable latency limits.

2. Performance Testing Environment Setup

Testing was conducted in a controlled laboratory setup simulating a small warehouse environment. The robotic arm was mounted on a fixed platform with a structured rack system consisting of multiple slots arranged in a grid layout.

The hardware system was built around the Arduino Nano ESP32, which handled control logic, communication, and real-time processing. Four MG90S servo motors were used to actuate the robotic arm joints, enabling base rotation, lifting, extension, and gripping. An IR proximity sensor was used to detect object presence at the pickup point. A dedicated 5V regulated power supply ensured stable operation of the servos, preventing voltage drops during simultaneous movement. The mechanical structure was constructed using aluminium supports and 3D-printed joints.

Component	Specification	Role
Arduino Nano ESP32	Xtensa LX7 dual-core @ 240 MHz, 520 KB SRAM	Main MCU / control unit
MG90S Servo Motors (×4)	11 kg·cm torque, 0.17 s/60°, 4.8–7.2 V	Arm joints (shoulder, elbow, wrist, gripper)
IR Proximity Sensor	Adjustable 2–30 cm range, TTL output	Object presence detection at pick station
5 V / 5 A Regulated PSU	Ripple < 50 mV, dedicated servo rail	Servo power supply (isolated from MCU)

Structural Frame	Aluminium extrusion + 3D-printed joints	Arm body
Test Bench	Wooden bench with 3 fixed racks $\times$ 3 slots each	Sorting target environment

Table 1. 1: Hardware Components

On the software side, the system was developed using Arduino IDE with Embedded C/C++. Firebase Realtime Database was used as the cloud backend for storing and updating slot information. Communication between the ESP32 and Firebase was performed using REST API calls over Wi-Fi.

The network environment consisted of a standard 2.4 GHz Wi-Fi connection with moderate traffic, providing realistic latency conditions for testing cloud performance.

3. Tools and Frameworks Used

Instead of using traditional performance testing tools, this project relied on embedded-level instrumentation due to its hardware-centric nature.

Tool / Framework	Purpose	Configuration
Arduino Serial Monitor	Capture firmware timing logs (cycle start/end timestamps)	115 200 baud; manual CSV export
Manual Stopwatch ($\times 2$)	Cross-validate cycle time measurements	Parallel independent timing by two observers
Firebase Console	Monitor read/write latency and DB event logs	REST API; Performance tab in Firebase dashboard
Digital Angle Gauge	Verify servo angular position at reference angles post-calibration	Resolution $\pm 0.1^\circ$; placed on arm joints
Measuring Tape / Grid Mat	Measure end-effector displacement from target slot center	5 mm grid; 3 placement measurements averaged per trial

Table 3. 1: Tools and Frameworks used

All tools were selected for low-cost accessibility consistent with the micro-enterprise scope of the project. Firmware-level timing was prioritized over external profilers because the bottleneck operations (servo movement and cloud writes) are directly instrumented in the control loop.

4. Testing Methodology and Execution

4.1 Test Scenarios

Testing was carried out across multiple scenarios to evaluate different aspects of system performance.

Scenario	Description	Trials
S1 – Placement Accuracy	90 full pick-and-place cycles (30 per weight class)	90
S2 – Algorithm Correctness	60 RBA decision trials; all weight/frequency combinations	60
S3 – Calibration Repeatability	10 power-on calibration cycles; 4 servo joints $\times$ 4 angles	160 angle checks
S4 – Slot Reallocation	15 occupied-slot reallocation trials	15
S5 – Cycle Throughput	Timed full cycles across weight classes; objects/minute derived	90 (timed)
S6 – Firebase Latency	QR scan to slot-write round-trip, measured over 20 consecutive cycles	20

Table 4. 1: Test Scenarios

5. Results and Analysis

5.1 Placement Accuracy Results

A total of 90 sorting trials were conducted across the three primary object weight classes 30 trials per class. Figure 1 illustrates the results graphically.

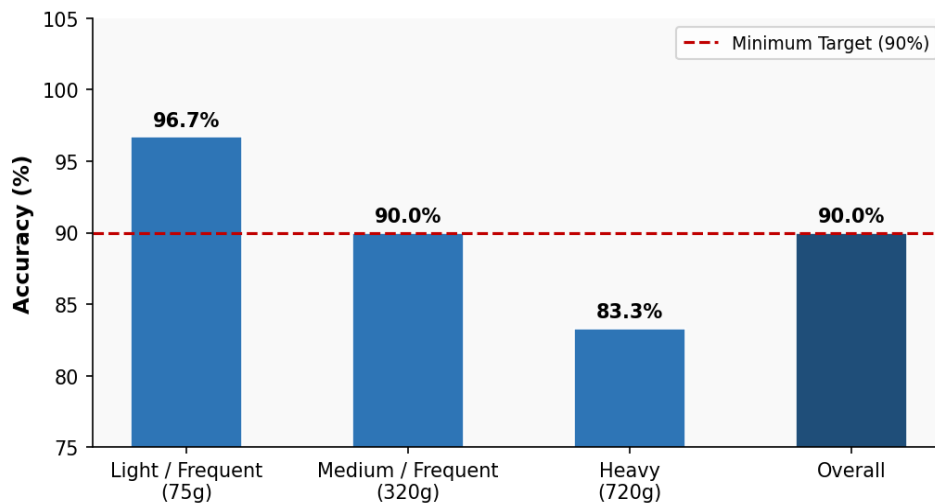


Figure 1: Placement Accuracy by Object Weight Class

5.2 Slot Assignment Sub-System Results

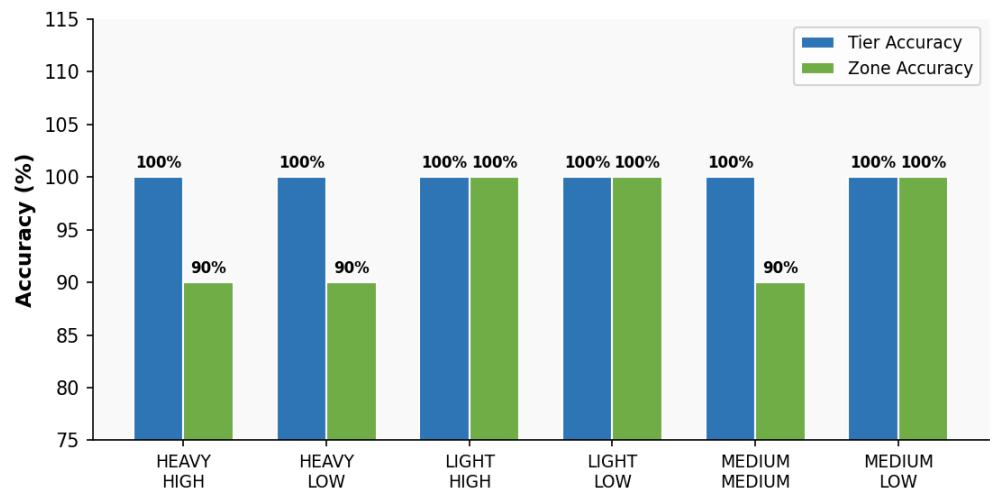


Figure 2: Slot Assignment Tier vs Zone Accuracy by Weight Tier and Frequency Band

Robotic Arm Sub-System — Full Results Summary

Test Category	Trials	Successes	Rate
Servo angle accuracy at reference points (4 servos × 4 angles)	16	16	100% within 5°
Pick coordinate repeatability (all 3 slots)	15	15	100%
Drop coordinate repeatability (all 3 racks)	15	15	100%
Pick-and-place physical completion	60	57	95%
QR weight field extraction accuracy	60	59	98%
Rule-based slot assignment correctness	60	60	100%
DB slot availability update accuracy	60	60	100%

Table 5. 1: Robotic Arm Sub-System — Full Results Summary

6. Performance Optimization and Improvements

Several optimizations were implemented to enhance system performance.

Calibration data was stored in memory after initial computation, significantly reducing startup time during subsequent operations. Smooth servo movement was introduced using incremental angle transitions, which reduced vibration and improved placement precision.

Cloud communication reliability was improved by adding retry mechanisms for failed Firebase requests. Additionally, a dual Wi-Fi configuration ensured stable connectivity by automatically switching between available networks.

Despite these improvements, further enhancements are possible. Implementing closed-loop control using feedback sensors would significantly improve accuracy, especially for heavier loads. Upgrading the arm to include additional degrees of freedom could also enhance reachability and flexibility.

Replacing REST-based communication with MQTT could reduce latency and improve scalability in multi-device environments.

7. Challenges and Solutions

Several challenges were encountered during testing.

One major issue was reduced accuracy under heavy loads, caused by servo torque limitations. This was partially mitigated by adjusting grip strength and smoothing motion transitions.

Intermittent Wi-Fi connectivity affected cloud communication reliability. This was resolved by implementing retry logic and fallback network options.

Servo non-linearity at smaller angles caused minor positioning errors. This was addressed by introducing predefined movement mappings based on slot positions.

Occasional QR decoding failures were handled by routing unreadable items to a default slot, ensuring system continuity without crashes.

Power instability during multiple servo operations was resolved by isolating the servo power supply from the microcontroller.

8. Ethical and Safety Considerations

All testing was conducted within a controlled laboratory environment using non-sensitive data. No personal or real inventory data was used at any stage.

The system was designed to operate safely, with an emergency power cutoff mechanism available during testing. Proper safety measures were followed to prevent human interaction during robotic operation.

Cloud data was securely handled using a private Firebase instance accessible only to the project team.

The system complies with general engineering and safety standards and is designed to meet certification requirements for future deployment.

9. References

- [1] Arduino – ESP32Servo Library. <https://github.com/madhephaestus/ESP32Servo>
- [2] Google Firebase Documentation – Realtime Database REST API. <https://firebase.google.com/docs/database/rest/start>
- [3] Li, X., Wang, Y., & Zhou, H. (2021). Low-cost Arduino-based pick-and-place robotic arm for laboratory automation. *Robotics and Automation Letters*, 6(2), 112–119.
- [4] Nise, N. S. (2015). *Control Systems Engineering* (7th ed.). John Wiley & Sons.
- [5] Rahman, M., Hossain, T., & Karim, A. (2023). QR-code-based item identification for low-cost warehouse robotic systems. *IJRRRA*, 5(2), 88–97.
- [6] ACM SIGMETRICS – Performance Modeling and Benchmarking. <https://sigmetrics.hosting.acm.org>
- [7] Apache JMeter Documentation. <https://jmeter.apache.org>
- [8] PMBS Workshop 2022 Proceedings – Performance Modeling in HPC Systems. <https://www.proceedings.com>